



Next Destination – A Transport Ticket Booking Web Application



Rikchhen Lama

230250

B.Sc.(Hons.) Computing, Softwarica College of IT and E-commerce, Coventry University

ST6003CEM Web API Development

Albert Maharjan

March 6, 2026

Table of Contents

Introduction.....	5
Aim and Objectives.....	5
Aim	5
Objectives	6
TypeScript strict mode to eliminate runtime errors in production	7
Technology Stack.....	9
REST API Development	10
Frontend Architecture	14
Design Pattern and Architecture	16
Challenges Faced and Lessons Learned	19
Skill Gained	19
Conclusion and Future Improvements	20
References	21
Appendix.....	22
Frontend GitHub Repository:	22
Backend GitHub Repository:	22
Project Demonstration Video:	22
Application Screenshots.....	23
Users UI	23
Business UI	27
Admin UI	30
ER Diagram	31
.....	31
Test logs	32
Backend Test Logs	32

Table of Figure

Figure 1: Research Session Info Graphic	8
Figure 2: Technology Stack Architecture	9
Figure 3: Rest API Request Follow	10
Figure 4: Admin Endpoints	11
Figure 5: User Endpoints	11
Figure 6: Booking Endpoints.....	12
Figure 7: Business Endpoints	12
Figure 8: Ticket Endpoints	12
Figure 9: Trip Endpoints.....	13
Figure 10: Next.js Frontend Architecture.....	14
Figure 11: Next.js App Router Project Structure used in NextDestination	15
Figure 12: Layered Backend Architecture Flow.....	17
Figure 13: Backend Folder Structure Implementation Layered Architecture	18
Figure 14: NextDestination Landing Page UI.....	23
Figure 15: NextDestination Login Page UI.....	23
Figure 16: : NextDestination Register Page UI.....	24
Figure 17: NextDestination Dashboard Page UI	24
Figure 18: NextDestination Ticket Page UI.....	25
Figure 19: NextDestination Booking Page UI	25
Figure 20: NextDestination View Ticket Page UI.....	26
Figure 21: NextDestination Edit Profile UI.....	26
Figure 22: Next Destination Business LogIn Page	27
Figure 23: Next Destination Business Dashboard UI.....	27
Figure 24: Next Destination Business Page All Ticket UI	28
Figure 25: Next Destination Business Create Ticket Page UI.....	28
Figure 26: Next Destination Busienss Approve Page UI	29
Figure 27: Next Destination Busienss Transaction Page UI	29
Figure 28 : Next Destination Admin Page Dashboard UI	30
Figure 29: ER Diagram	31
Figure 30: Admin Integration Test.....	32
Figure 31: Admin Unit Test.....	32
Figure 32: Booking Integration Test	33
Figure 33: Booking Unit Test	33
Figure 34: Business Integration Test	34
Figure 35: Business Unit Test.....	34
Figure 36: Ticket Integration Test.....	35

Figure 37: Ticket Unit Test.....	36
Figure 38: Trip Integration Test	36
Figure 39: Trip Unit Test	37
Figure 40: User Integration test.....	37
Figure 41: User Unit Test.....	38
Figure 42: Integration Test Log.....	38
Figure 43: Unit Test Log.....	38
Figure 44: Test Coverage Snippet.....	39

Introduction

NextDestination is a full-stack web application created for bus and plane ticket booking, with the goal of making the booking process more organised, secure, and convenient. The platform allows users to search for available trips, book tickets for one or more passengers, view their bookings, and manage their reservations. At the same time, it also supports business operators who are responsible for creating and managing trips, while administrators oversee business approval and overall platform activity.

The project was developed to solve common problems found in many transport booking systems, such as poor role management, lack of proper provider verification, and limited flexibility in the booking process. In NextDestination, businesses are required to register and submit their documents before they can start operating, and only approved businesses are allowed to create trips. This helps improve trust, reliability, and better control of the platform. Overall, the project shows how a modern TypeScript-based full-stack application can be used to address a real booking-related problem through secure API development and a well-structured connection between the frontend and backend.

Aim and Objectives

Aim

The aim of this project is to develop NextDestination, a full-stack bus and plane ticket booking system that helps users search available trips, book tickets, manage reservations, and view their generated tickets. It also provides verified business operators with the ability to add and manage trips through a secure role-based platform.

Objectives

The objective of the project are to:

- To develop a secure bus and plane ticket booking system for users, businesses, and administrators.
- To implement a business verification process so that only approved businesses can create and manage trips.
- To build trip management features that allow verified businesses to add, update, and manage travel schedules.
- To create a booking system that lets users book tickets for multiple passengers and manage their reservations.
- To integrate a full-stack TypeScript architecture using Next.js, Node.js, Express, and MongoDB for a reliable and maintainable web application.

TypeScript strict mode to eliminate runtime errors in production

An important technical aspect of this project was the use of TypeScript strict mode to make the code more reliable. It would not be correct to say that strict mode completely prevents runtime errors, but it does help reduce many avoidable mistakes by identifying them earlier during development. This was especially useful in NextDestination because the system handles different user roles, nested booking information, authentication states, and frequent data exchange between the frontend and backend.

Strict typing was particularly helpful in booking-related features, where one user can enter details for several passengers in a single request. Each passenger may include information such as name, age, gender, and seat number, so there is a higher chance of data being incomplete or incorrectly structured if the system is not carefully checked. In a loosely typed setup, these kinds of issues are easier to overlook. With TypeScript strict mode, many of these unsafe assumptions can be caught before the code actually runs. The same benefit was seen in authentication logic, where nullable tokens, optional user information, and role-based checks could easily lead to errors if they were not handled properly.

Another advantage was the consistency it brought across the whole project. Since both the frontend and backend were written in TypeScript, it was easier to keep the system understandable and maintainable. API handlers, helper functions, and route-related logic became clearer to read, and the purpose of the code was easier to follow. At the same time, this approach was not without difficulties. Using strict mode requires more careful coding and can sometimes make the code longer, especially when dealing with optional values or external library types. Even with that extra effort, it was still a valuable decision because this project needed stability and maintainability more than quick development alone.

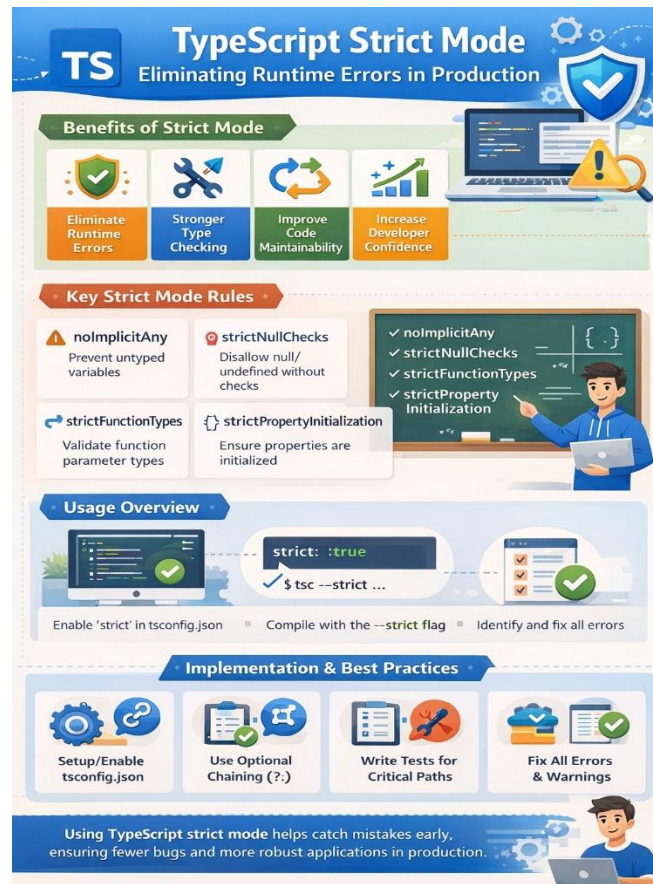


Figure 1: Research Session Info Graphic

Technology Stack

NextDestination was developed using Next.js and TypeScript for the frontend, Node.js with Express and TypeScript for the backend, and MongoDB as the database. This technology stack was chosen because it is modern, flexible, and well suited for full-stack web application development.

Next.js was used to build the user interface and manage routing, while Express was used to create the RESTful APIs. MongoDB was selected to store data such as users, trips, bookings, and tickets. TypeScript helped improve code reliability on both the frontend and backend, and Jest with SuperTest was used to test backend functionality.



Figure 2: Technology Stack Architecture

REST API Development

The backend of NextDestination was developed as a RESTful API that supports the main roles in the system, including users, businesses, and administrators. It handles important features such as authentication, business approval, trip management, booking, and ticket processing, using JWT with cookies for secure access.

Approved businesses are allowed to create and manage trips, while users can make bookings for multiple passengers in a single request. Once a booking is completed, tickets are generated, and the system also includes partly implemented QR-based features for scanning and voiding tickets.

To make the backend more reliable, key API routes were tested using Jest and SuperTest.



Figure 3: Rest API Request Follow

The Following table illustrate the main endpoints used in the application.

Base Path: api/admin		
Method	Endpoint	Description
GET	/admin/users	Get all Users in Application
GET	/admin/users/phone	Get all Users By PhoneNumber
GET	/admin/users/id	Get all Users By ID
DEL	/admin/users/delete	Delete user
DEL	/admin//users/delete/delete all	Delete all users
POST	/user/login	LogIn as Admin
GET	/admin/user/edit	Edit users
PUT	/admin/approve-business	Approve Business

Figure 4: Admin Endpoints

Base Path: /user		
Method	Endpoint	Description
POST	/user/register	Register all Users
POST	/users/login	Login all Users
PATCH	/users/me	Edit Users Operations
GET	/users/mr	Get User Profile
POST	/user/request-password-reset	Resest Password

Figure 5: User Endpoints

Base Path: /booking		
Method	Endpoint	Description
POST	/booking/create	Create Booking
GET	/booking/mine	Get all Bookings
GET	/booking/id	Get Booking by ID
GET	/booking/ref	Get Booking by ref

Figure 6: Booking Endpoints

Base Path: /business		
Method	Endpoint	Description
POST	/business/register	Register all Business
POST	/business/login	Login all Business
POST	/business/upload-document	Upload Business Document

Figure 7: Business Endpoints

Base Path: /ticket		
Method	Endpoint	Description
GET	/ticket/booking	Get Ticket by Booking
GET	/ticket/id	Get Tickets byID
GET	/ticket/scan	Scan Tickets
PATCH	/ticket/void	Void Tickets
PATCH	/ticket/cancel	Cancel Bookings

Figure 8: Ticket Endpoints

Base Path: /trip		
Method	Endpoint	Description
POST	/trip/create	Create Trip
GET	/trip/business/mine	Get all BUiness By Tickets
GET	/trip/search	Search Trip
GET	/trip/id	Get Trip by ID
GET	/trip/business/mine	Get Business Trip
PATCH	/trip/edit	Update Trip

Figure 9: Trip Endpoints

Frontend Architecture

The frontend of NextDestination was built with Next.js and TypeScript to support the main users of the system, including customers, business operators, and administrators. It covers essential parts of the application such as login and registration, protected pages, trip viewing, and booking-related actions. These parts were developed using reusable components and structured communication with the backend.

For API communication, the project uses separate handler files for different features. This made the code easier to organise because each feature manages its own requests instead of keeping everything together in one place. A shared Axios setup was also used so common request settings could be reused throughout the application.

User authentication is handled with JWT stored in cookies, and route access is controlled through middleware in the main proxy file. This helps the system check whether a user is logged in and whether they have the correct role to open certain pages. Because of this, admin pages stay restricted, and logged-in users are redirected away from pages like login and registration.

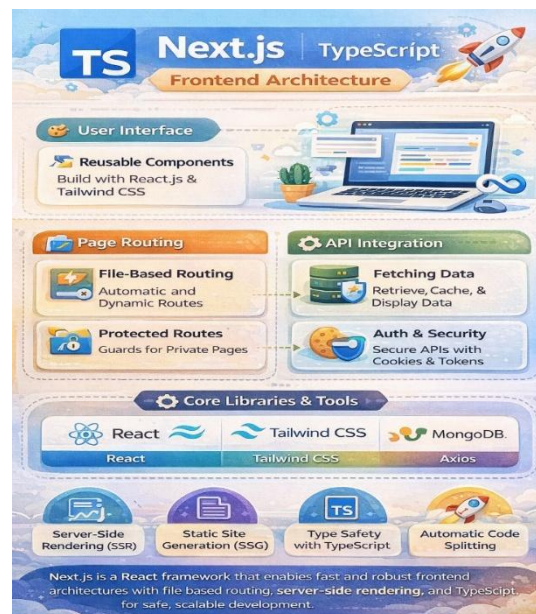


Figure 10: Next.js Frontend Architecture

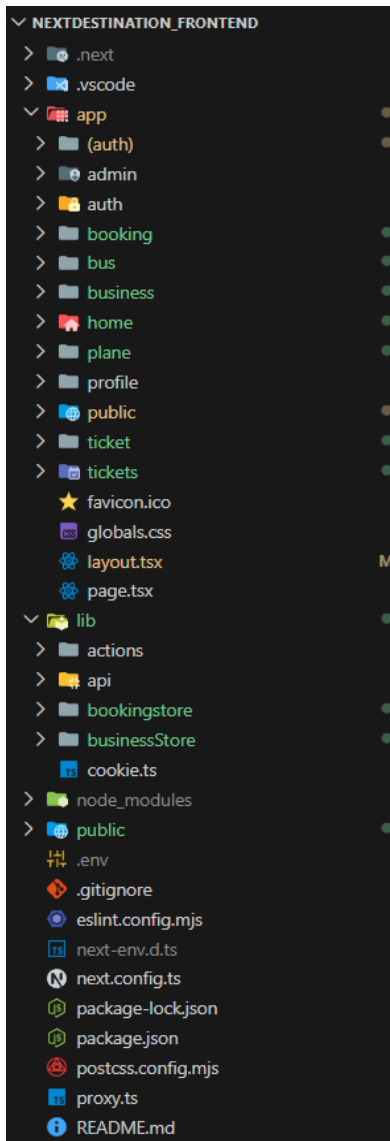


Figure 11: Next.js App Router Project Structure used in NextDestination

Design Pattern and Architecture

NextDestination follows a layered architecture on the backend, where the code is divided into models, types, DTOs, repositories, services, controllers, and routes. This structure helped keep the project organised by giving each layer a clear responsibility. For example, routes handle endpoint definitions, controllers manage requests and responses, services contain the main business logic, repositories deal with database operations, and DTOs help control the shape of data passed through the system.

This design made it easier to manage important features such as authentication, business verification, trip creation, booking, and ticket handling without mixing everything into one place. On the frontend, a similar modular approach was followed by separating UI components, API handler files, cookie utilities, and route protection logic. This improved readability and made the application easier to maintain as the project grew.

Overall, the system works as a multi-role booking platform with a clear flow: business registration, admin approval, trip creation, user booking, ticket generation, and ticket validation. This made the project more practical than a simple CRUD-based system, because each part depends on the previous stage and follows a more realistic workflow.

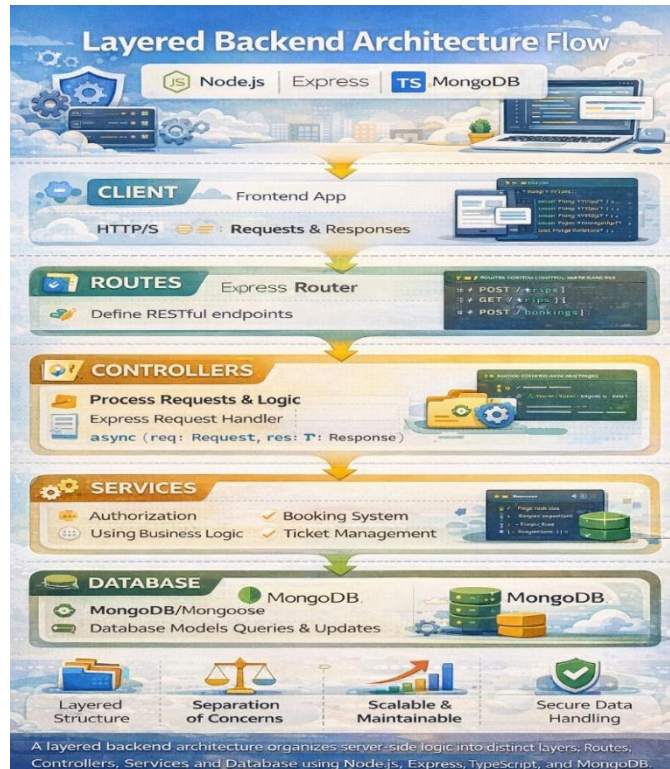


Figure 12: Layered Backend Architecture Flow

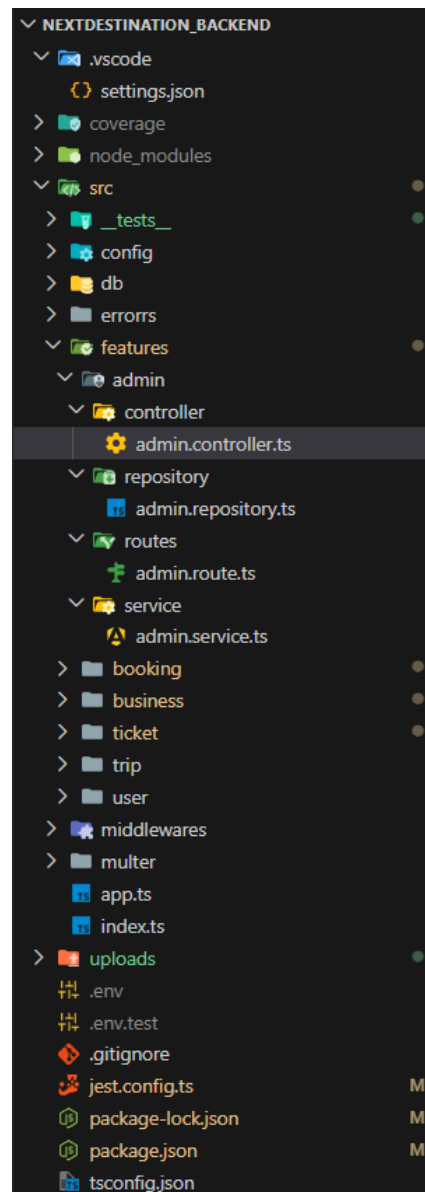


Figure 13: Backend Folder Structure Implementation Layered Architecture

Challenges Faced and Lessons Learned

One of the main challenges during this project was getting the frontend and backend to work smoothly together, especially when dealing with authentication and route protection. I had to be careful with how cookies, API requests, and access control were handled so that each user could only open the pages meant for their role.

Another challenge was building the business approval and booking flow in a way that matched a real system. Businesses that were not verified had to be restricted from creating trips, while regular users still needed a simple booking process for multiple passengers. Working on these parts helped me understand the importance of keeping the code well organised, validating data properly, and testing features step by step throughout development.

Skill Gained

This project contributed to growth in both technical and non-technical areas. On the technical side, it strengthened understanding of REST API design, full-stack development with TypeScript, authentication using JWT and cookies, reusable API handler structures, and backend testing with Jest and SuperTest. It also improved understanding of how database design, business rules, and user flows connect within a full-stack application.

On the non-technical side, the project supported the development of problem-solving, planning, and decision-making skills. It also highlighted that software development is not only about adding features, but also about making architectural decisions that keep a system organised, scalable, and easier to maintain over time.

Conclusion and Future Improvements

NextDestination demonstrates how a modern TypeScript-based full-stack technology stack can be used to create a practical transport booking platform. The project brings together important features such as role-based access control, business verification, trip management, multi-passenger booking, and ticket generation within a single structured web application.

In the future, the system could be further improved by adding payment integration, fully completing QR-based ticket validation, and enhancing error handling and overall user experience. Overall, the project was a valuable learning experience that helped strengthen both web API development knowledge and full-stack application design skills.

References

Express - Node.js web application framework. (n.d.). <https://expressjs.com/>

Express routing. (n.d.). <https://expressjs.com/en/guide/routing.html>

Getting started · jest. (2025, June 10). <https://jestjs.io/docs/getting-started>

Team, M. D. (n.d.). *What is MongoDB? Database Manual - MongoDB Docs.* <https://www.mongodb.com/docs/manual/>

Vercel. (n.d.). *Next.js docs.* Next.js. <https://nextjs.org/docs>

Node.js — Introduction to Node.js. (n.d.). <https://nodejs.org/en/learn/getting-started/introduction-to-nodejs>

TSCConfig Reference - Docs on every TSCConfig option. (n.d.). <https://www.typescriptlang.org/tsconfig/#strict>

Documentation - the basics. (n.d.). <https://www.typescriptlang.org/docs/handbook/2/basic-types.html>

Auth. (2024, November 30). *JSON Web Token Introduction - JWt.io.* JSON Web Tokens - jwt.io. <https://www.jwt.io/introduction>

Axios API | Axios Docs. (n.d.). https://axios-http.com/docs/api_intro

Handling Errors | Axios Docs. (n.d.). https://axios-http.com/docs/handling_errors

Interceptors | Axios Docs. (n.d.). <https://axios-http.com/docs/interceptors>

Authentication - OWASP Cheat Sheet Series. (n.d.). https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html

Vercel. (2026, February 27). *File-system conventions: proxy.js.* Next.js. <https://nextjs.org/docs/app/api-reference/file-conventions/proxy>

Vercel. (2026b, February 27). *Next.js docs: App router.* Next.js. <https://nextjs.org/docs/app>

Mongoose v9.2.4: Validation. (n.d.). <https://mongoosejs.com/docs/validation.html>

Documentation - TypeScript 4.4. (n.d.). <https://www.typescriptlang.org/docs/handbook/release-notes/typescript-4-4.html>

Documentation - everyday types. (n.d.).
<https://www.typescriptlang.org/docs/handbook/2/everyday-types.html>

Mongoose v9.2.4: Query population. (n.d.). <https://mongoosejs.com/docs/populate.html>

Mongoose v9.2.4: Middleware. (n.d.). <https://mongoosejs.com/docs/middleware.html>

Appendix

Frontend GitHub Repository:

https://github.com/Rikchhen/NextDestination_web_Frontend.git

Backend GitHub Repository:

https://github.com/Rikchhen/NextDestination_Backend.git

Project Demonstration Video:

<https://youtu.be/JaS0RKxN2Jw>

Application Screenshots

Users UI

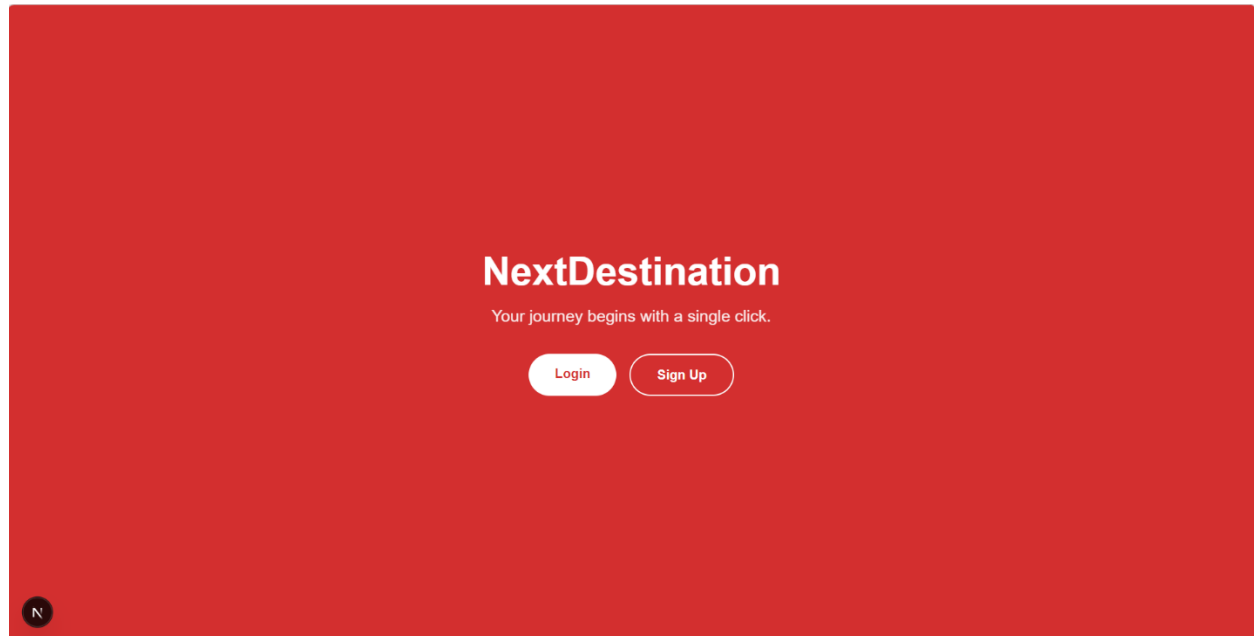


Figure 14: NextDestination Landing Page UI

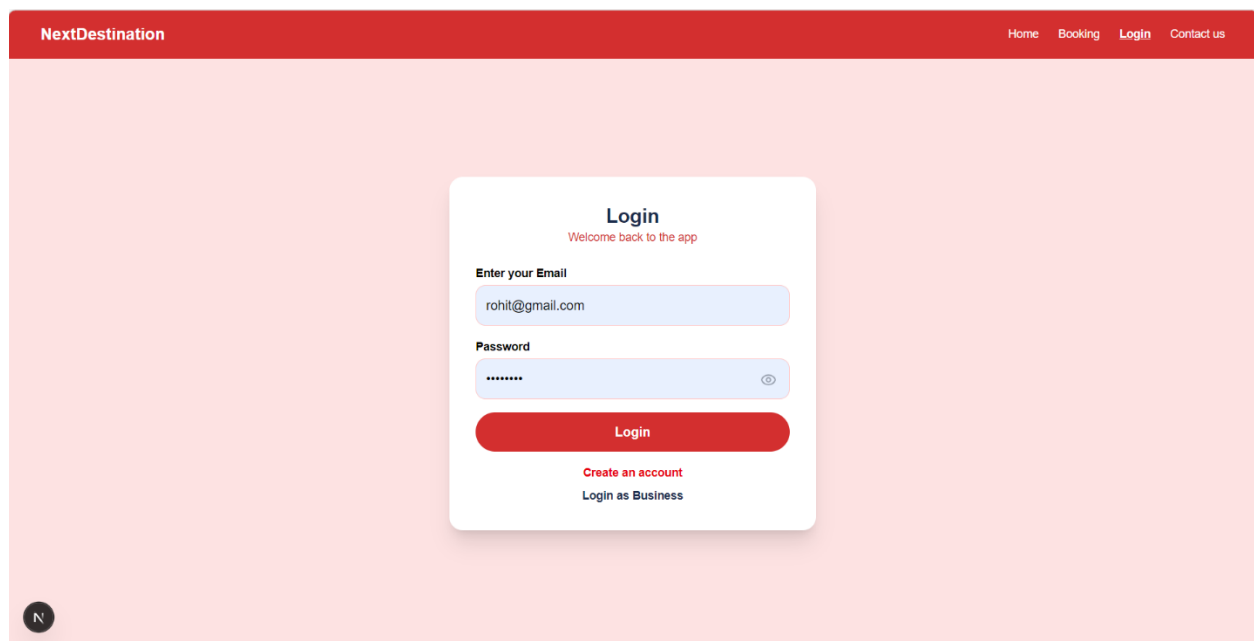


Figure 15: NextDestination Login Page UI

NextDestination
Home
Booking
Login
Contact us

Create an account

Name

Phone Number

Email

Password

Confirm Password

By continuing, you agree to our [terms of service](#).

Sign up

Already have an account? [Sign in here](#)

Figure 16: : NextDestination Register Page UI

HB

Hari Bahadur

Select your destination

LIBRARY

Home

My Tickets

Contact us

Booking

About Us

Logout

NextDestination
travels & buses easily

Select Your Destination

✈️

🚌

Airplane Tickets

See all >

LOGO

Buddha Air

NPR 6647

NPT 07:30 25Min NPT 07:55

Class Y | ATR | 25KG

25Min

FLIGHT DETAILS

Refundable

LOGO

Yeti Airlines

NPR 7452

NPT 07:30 25Min NPT 07:55

Class Y | ATR | 25KG

25Min

FLIGHT DETAILS

Refundable

Bus Tickets

See all >

Golden Super Delux

NPR 1200

A/C Sleeper (2+2)

9:00 AM — 9:45 AM

45 Min

15 Seats left

Nepal Travels

NPR 1400

A/C Sleeper (2+2)

9:00 AM — 9:20 AM

20 Min

7 Seats left

Figure 17: NextDestination Dashboard Page UI

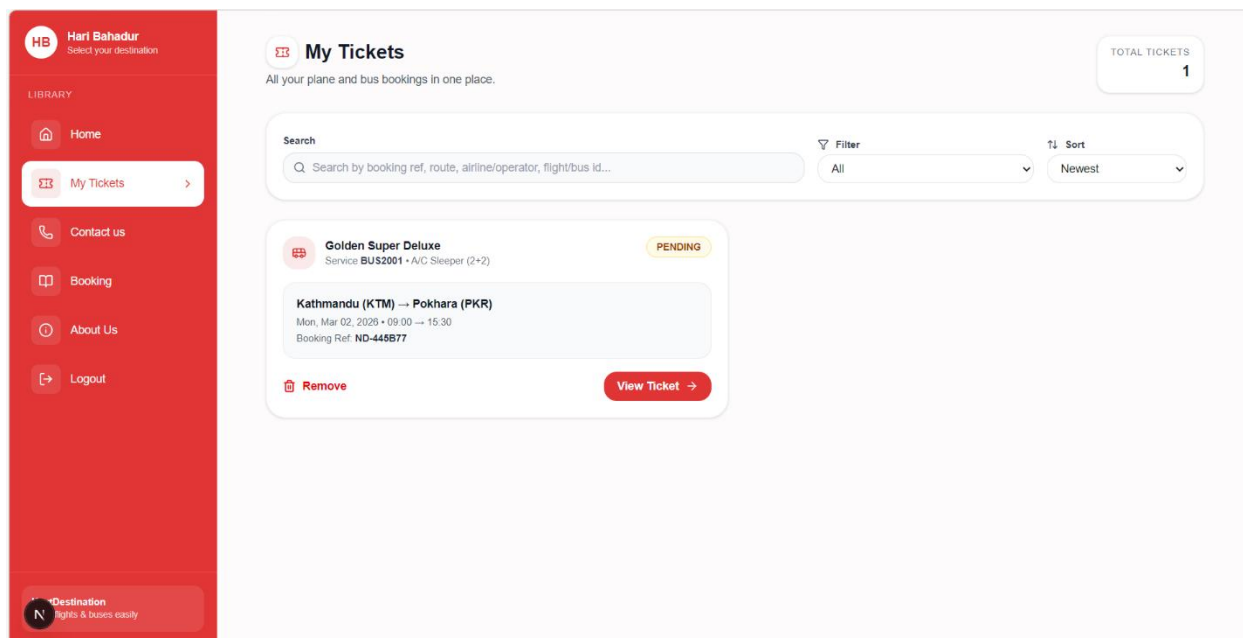


Figure 18: NextDestination Ticket Page UI

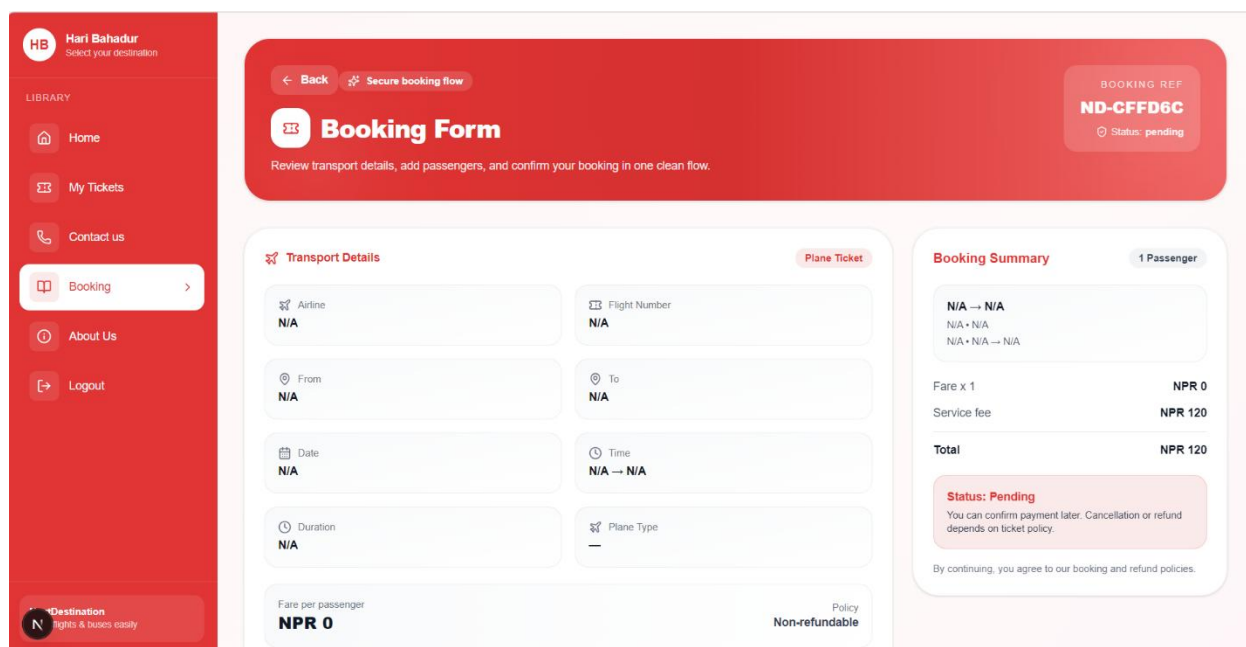


Figure 19: NextDestination Booking Page UI

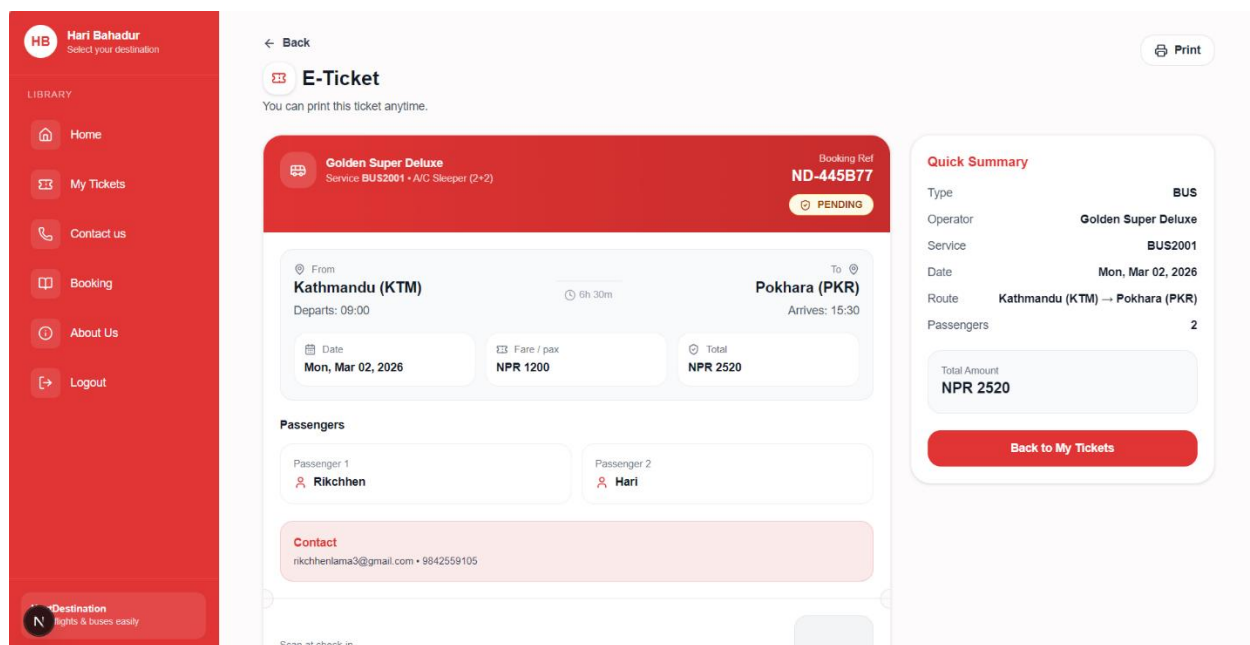


Figure 20: NextDestination View Ticket Page UI

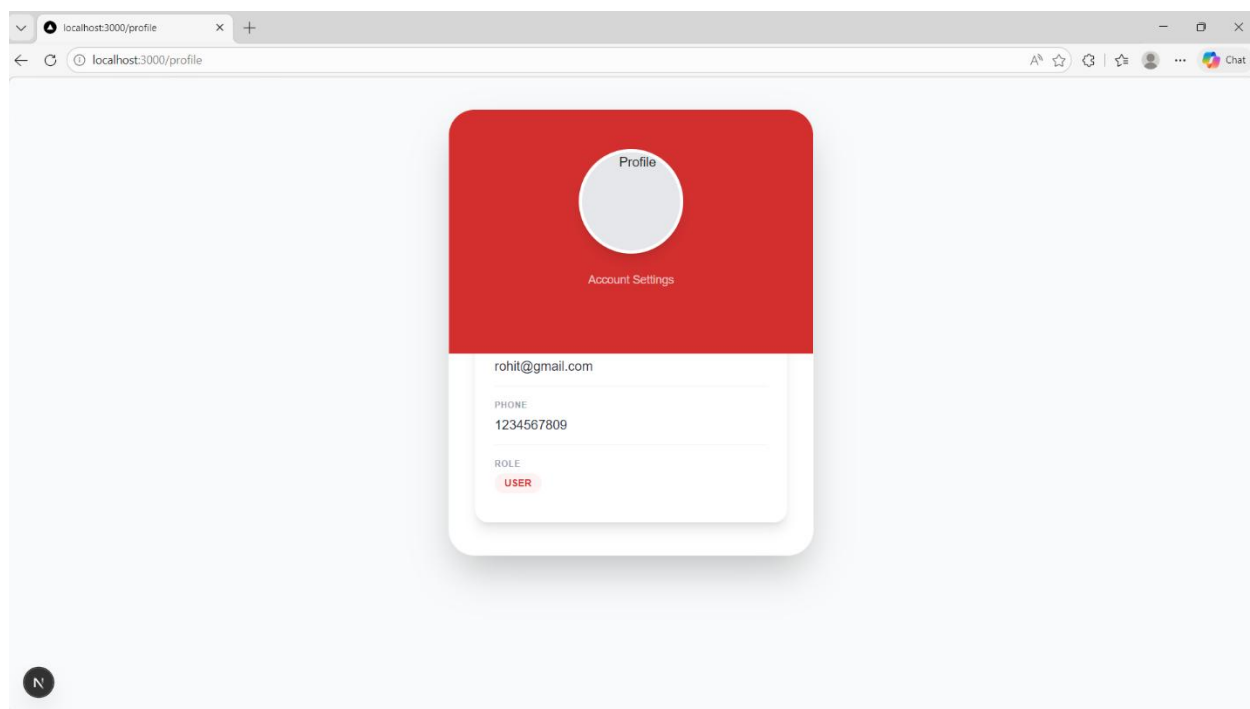


Figure 21: NextDestination Edit Profile UI

Business UI

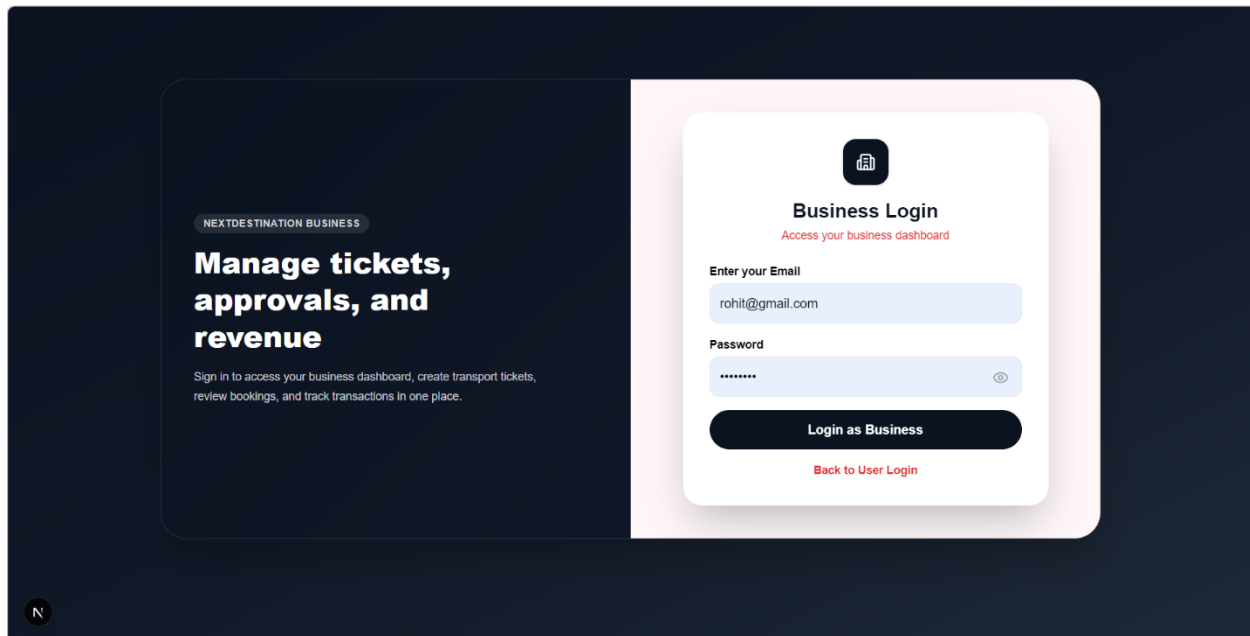


Figure 22: Next Destination Business Login Page

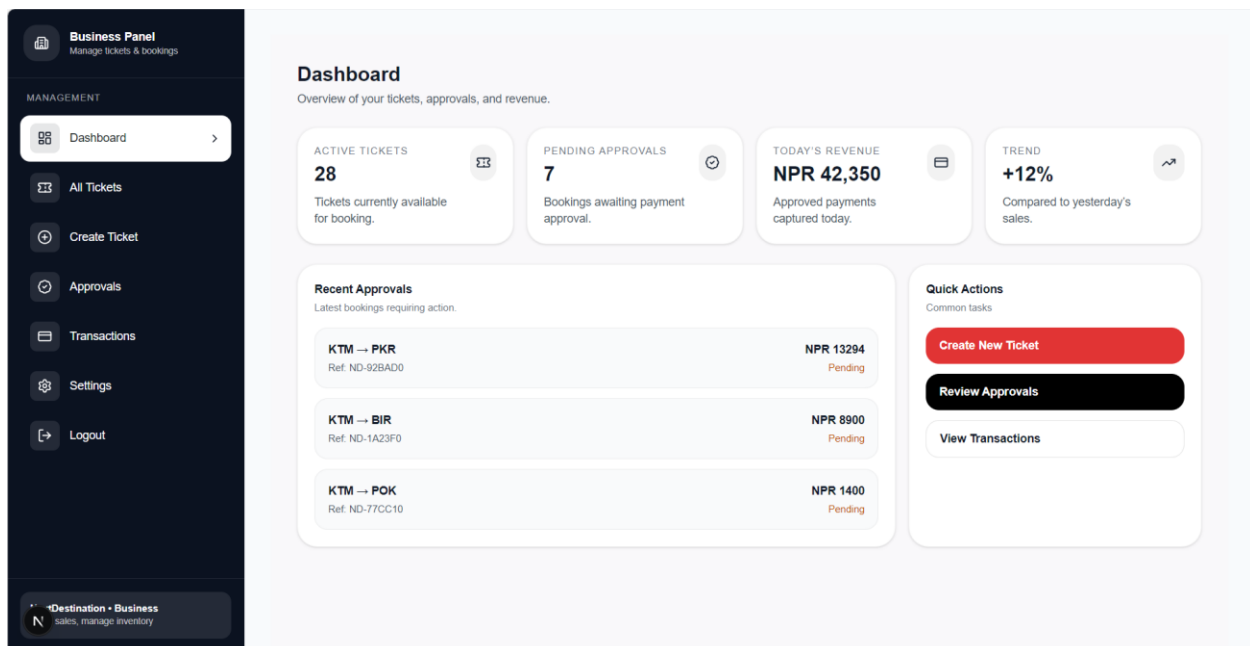


Figure 23: Next Destination Business Dashboard UI

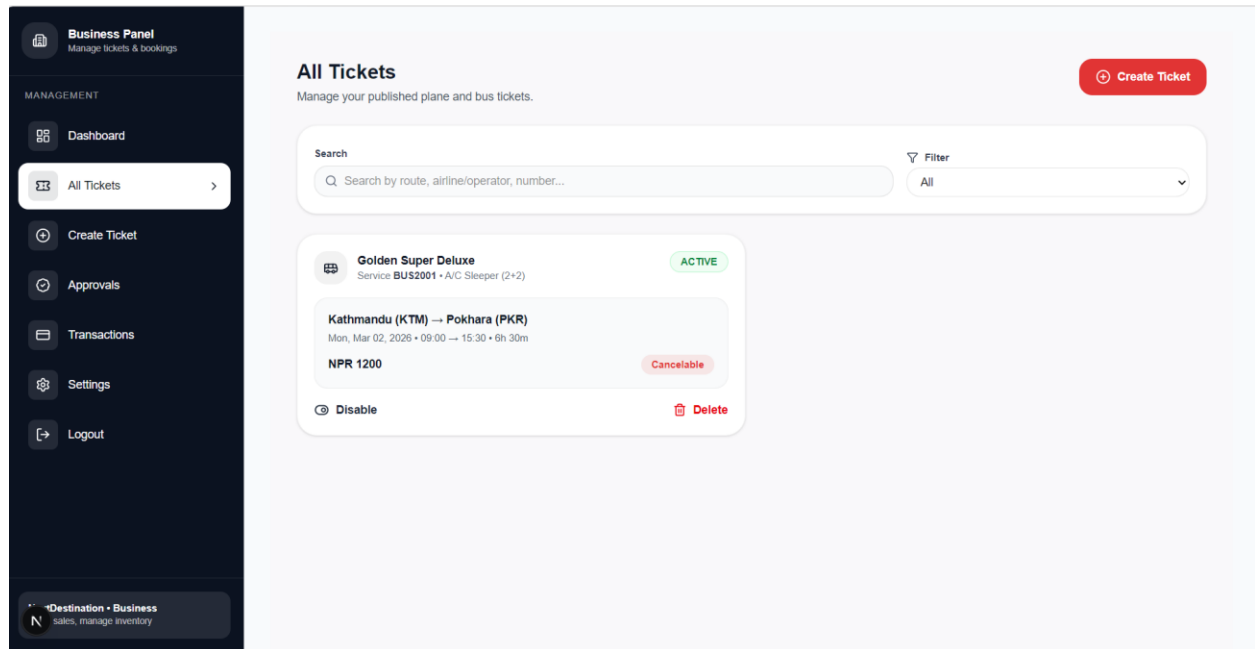


Figure 24: Next Destination Business Page All Ticket UI

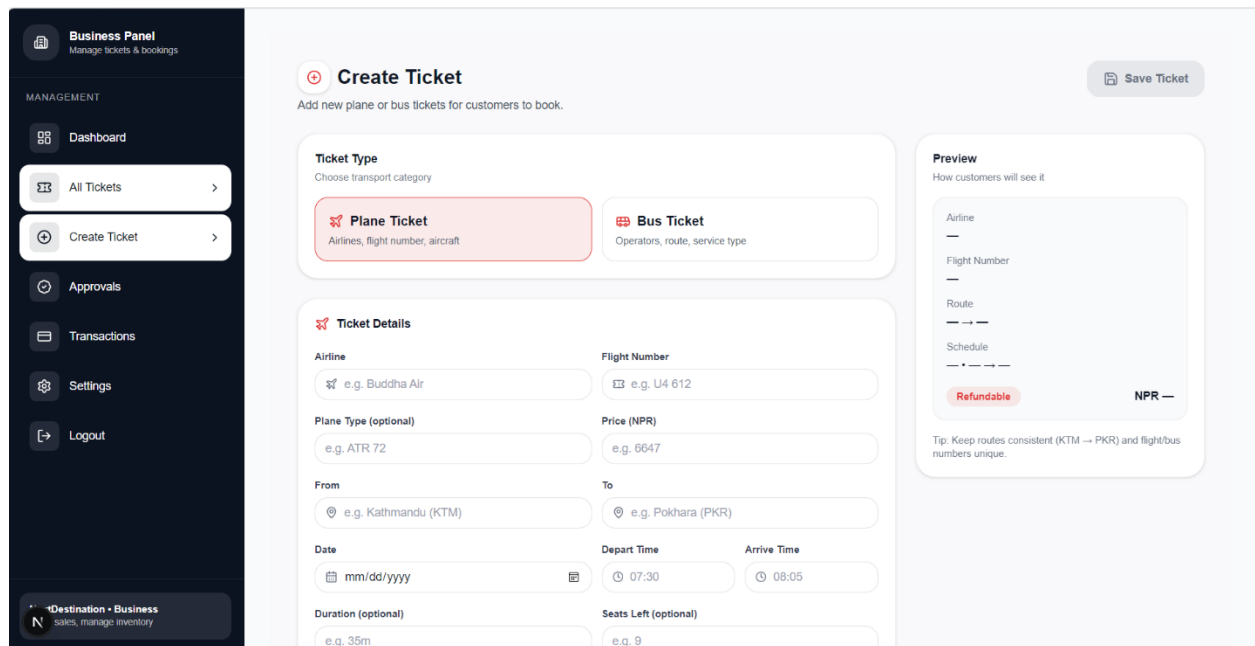


Figure 25: Next Destination Business Create Ticket Page UI

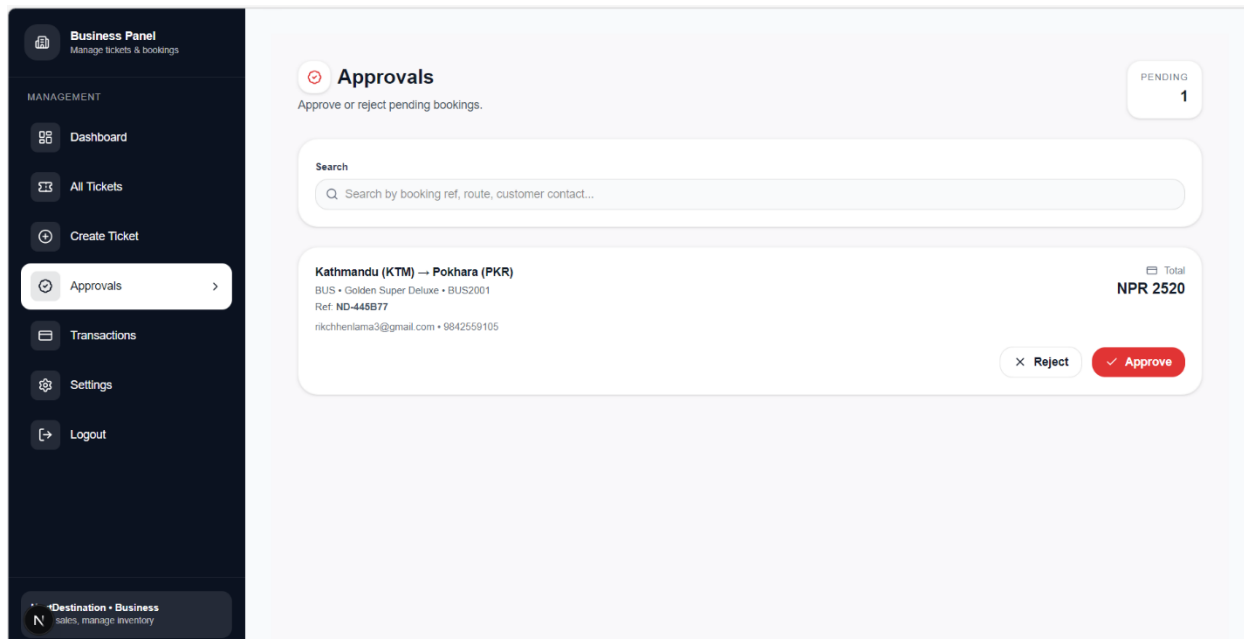


Figure 26: Next Destination Business Approve Page UI

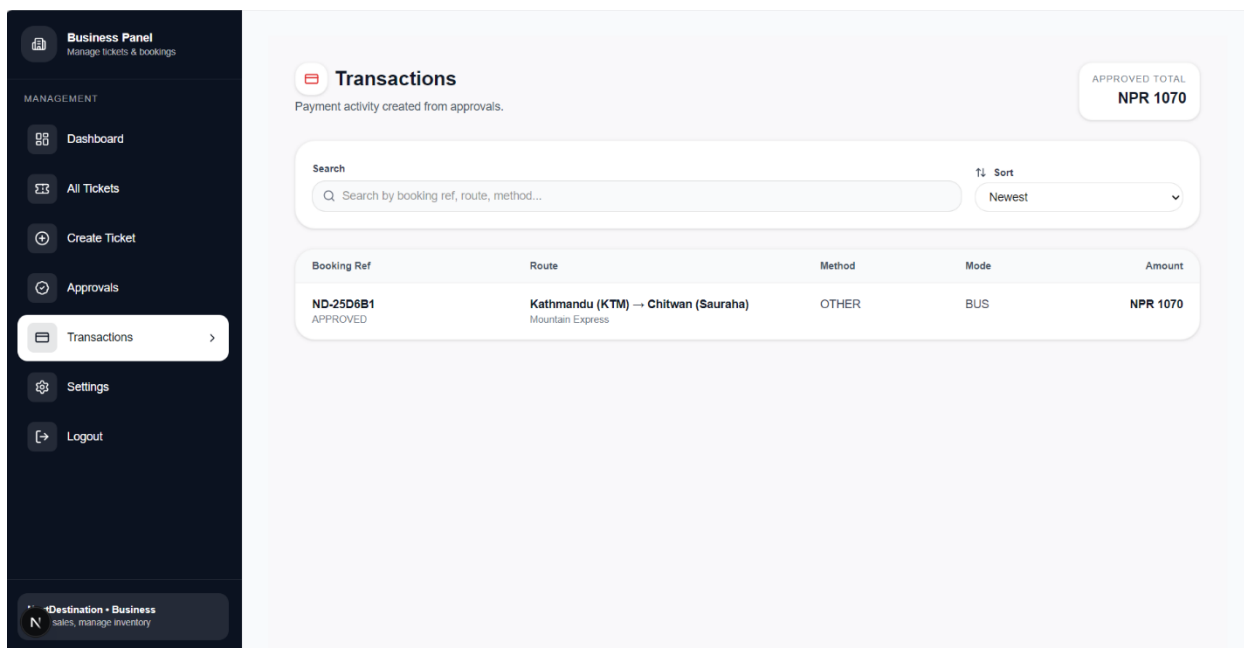


Figure 27: Next Destination Business Transaction Page UI

Admin UI

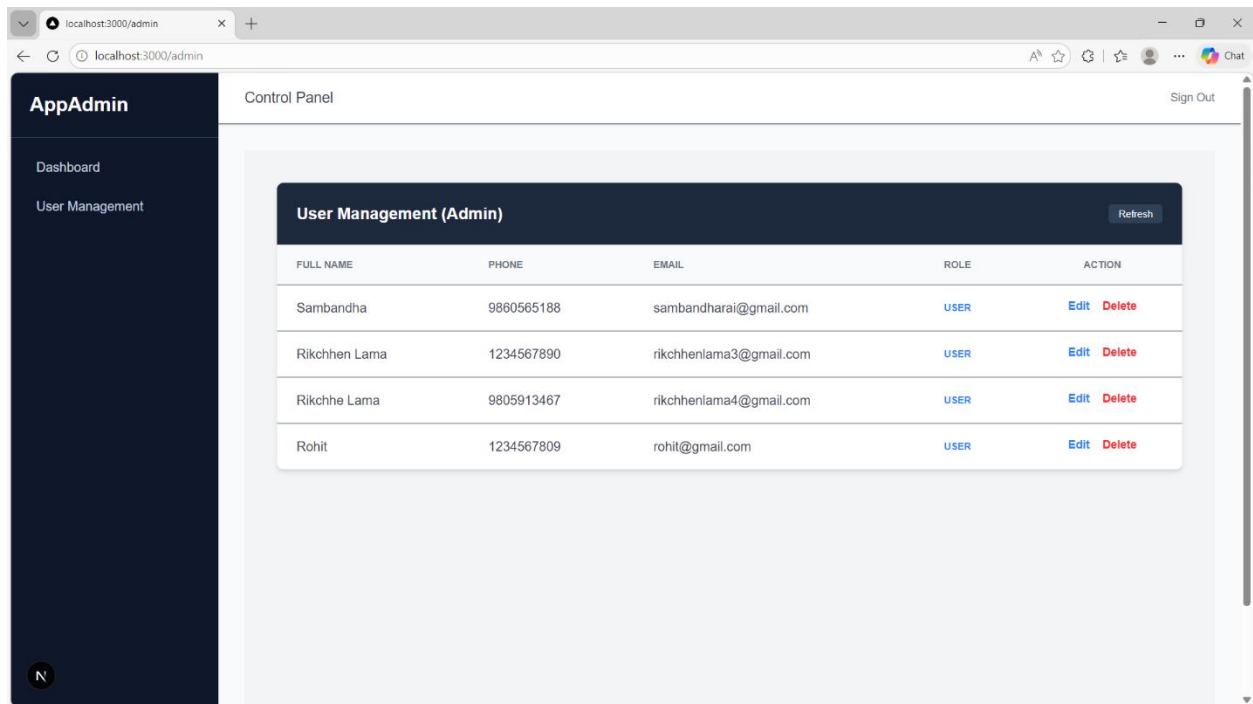


Figure 28 : Next Destination Admin Page Dashboard UI

ER Diagram

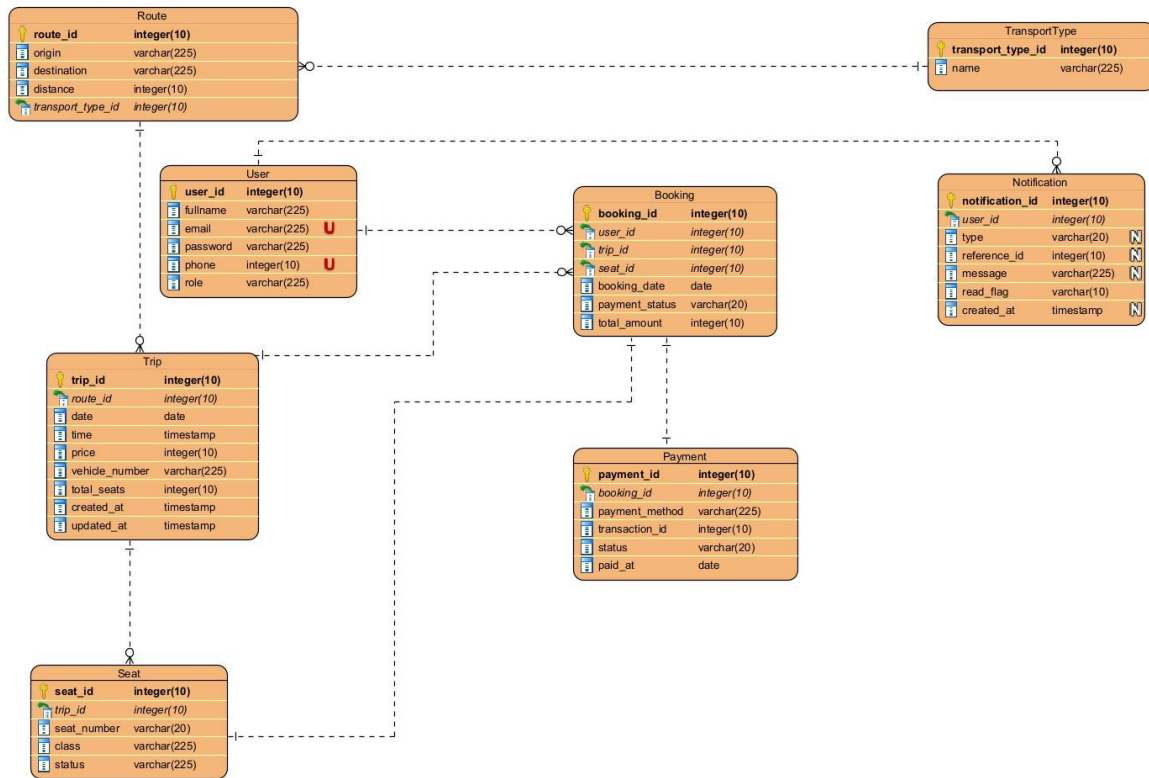


Figure 29: ER Diagram

Test logs

Backend Test Logs

```

PASS integration src/__tests__/integration/admin/admin.test.ts
Admin Integration Tests
  ✓ 1) GET /users -> 401/403 without token (52 ms)
  ✓ 2) GET /users -> 403 if not admin (32 ms)
  ✓ 3) GET /users -> 200 for admin and returns list (27 ms)
  ✓ 4) GET /users/id/:userId -> 200 for admin (15 ms)
  ✓ 5) GET /users/id/:userId -> 404/400 for invalid id (12 ms)
  ✓ 6) GET /users/phone/:phoneNumber -> 200 for existing phone (12 ms)
  ✓ 7) GET /users/phone/:phoneNumber -> 404/400 if not found (10 ms)
  ✓ 8) PATCH /users/edit/:userId -> 403 for non-admin (20 ms)
  ✓ 9) PATCH /users/edit/:userId -> 200 for admin (or 400 if controller validates) (37 ms)
  ✓ 10) DELETE /users/delete/:userId -> 403 for non-admin (21 ms)
  ✓ 11) DELETE /users/delete/:userId -> 200 for admin (23 ms)
  ✓ 12) PUT /approve-business/:businessId -> 200 for admin approve (16 ms)

Test Suites: 1 passed, 1 total
Tests:       12 passed, 12 total
Snapshots:   0 total
Time:        3.309 s
Ran all test suites matching admin.test.ts.

```

Figure 30: Admin Integration Test

```

Running one project: unit
PASS unit src/__tests__/unit/admin/admin.service.test.ts
AdminService unit tests
  ✓ getAllUsers: uses skip/limit and sanitizes users (3 ms)
  ✓ getUserById: throws if not found (5 ms)
  ✓ getUserById: returns sanitized user (1 ms)
  ✓ getUserByPhoneNumber: throws if not found (1 ms)
  ✓ getUserByEmail: throws if not found
  ✓ deleteUser: throws if user missing
  ✓ deleteUser: deletes and returns message
  ✓ deleteAllUsers: calls repo and returns message (1 ms)

Test Suites: 1 passed, 1 total
Tests:       8 passed, 8 total
Snapshots:   0 total
Time:        2.614 s

```

Figure 31: Admin Unit Test


```

PASS integration src/__tests__/integration/booking/booking.test.ts
Booking Integration Tests
  ✓ 1) POST /create -> 401 without token (18 ms)
  ✓ 2) POST /create -> 400 invalid DTO (13 ms)
  ✓ 3) POST /create -> 400 Trip not found (28 ms)
  ✓ 4) POST /create -> 400 Trip is not active (cancelled) (21 ms)
  ✓ 5) POST /create -> 400 seat already booked (50 ms)
  ✓ 6) POST /create -> 400 Not enough seats available (22 ms)
  ✓ 7) GET /:id -> 200 owner can view (19 ms)
  ✓ 8) GET /:id -> 404 for non-owner (Not allowed) (19 ms)
  ✓ 9) GET /:id -> 200 admin can view (22 ms)
  ✓ 10) GET /ref/:ref -> 200 business can view (17 ms)
  ✓ 11) PATCH /cancel/:id -> 200 owner cancels, tickets void, seats restored (105 ms)
  ✓ 12) POST /create -> 400 duplicate seat numbers in same request (DTO refine) (11 ms)

Test Suites: 1 passed, 1 total
Tests:       12 passed, 12 total
Snapshots:   0 total
Time:        4.242 s
Ran all test suites matching booking.test.ts.

```

Figure 32: Booking Integration Test

```

Running one project: unit
PASS unit src/__tests__/unit/booking/booking.service.test.ts
BookingService unit tests
  ✓ createBooking: throws Trip not found (7 ms)
  ✓ createBooking: throws Trip is not active (1 ms)
  ✓ createBooking: throws Seats already booked (1 ms)
  ✓ createBooking: throws Not enough seats available when atomic update fails (1 ms)
  ✓ createBooking: success creates booking and tickets (2 ms)
  ✓ createBooking: ticket creation failure rolls back seats and cancels booking (1 ms)
  ✓ getBookingById: blocks non-owner non-privileged (1 ms)
  ✓ cancelBooking: throws if already cancelled (1 ms)

Test Suites: 1 passed, 1 total
Tests:       8 passed, 8 total
Snapshots:   0 total
Time:        2.57 s
Ran all test suites matching booking.service.test.ts.

```

Figure 33: Booking Unit Test

```

PASS integration src/__tests__/integration/business/business.test.ts
Business Integration Tests
  POST /api/business/register
    ✓ 1) should fail when body invalid (400) and return errors (87 ms)
    ✓ 2) should fail when profile picture missing (400 File Not Inserted) (34 ms)
    ✓ 3) should register successfully with file upload (201) (131 ms)
    ✓ 4) should fail when business already exists (400) (20 ms)
  POST /api/business/login
    ✓ 5) should fail when DTO invalid (400) with errors (27 ms)
    ✓ 6) should fail when business not found (400) (18 ms)
    ✓ 7) should fail when wrong password (>= 6 chars) (400 Invalid credentials) (78 ms)
    ✓ 8) should block login when Pending and no document (400) (72 ms)
  POST /api/business/upload-document
    ✓ 9) should reject upload without auth token (401) (19 ms)
    ✓ 10) should reject upload with token but no file (400 No document uploaded) (22 ms)
    ✓ 11) should upload document successfully (200) (44 ms)
  PUT /api/business/admin/approve/:businessId
    ✓ 12) should reject approve without auth (401/403) (12 ms)
    ✓ 13) should reject approve if not admin (403) (17 ms)
    ✓ 14) should approve successfully as admin (200) (22 ms)

Test Suites: 1 passed, 1 total
Tests:       14 passed, 14 total
Snapshots:   0 total
Time:        4.73 s
Ran all test suites matching business.test.ts.

```

Figure 34: Business Integration Test

```

PASS unit src/__tests__/unit/business/business.service.test.ts
BusinessService unit tests
  ✓ register: throws if business already exists (7 ms)
  ✓ register: hashes password, creates business, returns tempToken + sanitized business (3 ms)
  ✓ login: throws if business not found (3 ms)
  ✓ login: throws invalid credentials if password mismatch (3 ms)
  ✓ login: blocks when Pending and no document uploaded (3 ms)
  ✓ login: blocks when Rejected (2 ms)
  ✓ login: returns token + sanitized business when allowed (3 ms)
  ✓ uploadDocument: throws if business not found (2 ms)
  ✓ uploadDocument: sets businessDocument + Pending and saves (1 ms)
  ✓ approveBusiness: sets Approved + verified true (1 ms)
  ✓ approveBusiness: sets Rejected + stores reason (default if none)
  ✓ getAllBusinesses: uses skip/limit and sanitizes (1 ms)
  ✓ editBusinessProfile: does not update password/email and returns sanitized output (1 ms)

Test Suites: 1 passed, 1 total
Tests:       13 passed, 13 total
Snapshots:   0 total
Time:        1.442 s
Ran all test suites matching business.service.test.ts.

```

Figure 35: Business Unit Test

```

PASS integration src/__tests__/integration/ticket/ticket.test.ts
Ticket Integration Tests
  ✓ 1) GET /api/ticket/:id -> 401 without token (32 ms)
  ✓ 2) GET /api/ticket/:id -> 404 when ticket not found (31 ms)
  ✓ 3) GET /api/ticket/:id -> 200 owner can view (17 ms)
  ✓ 4) GET /api/ticket/:id -> 404 non-owner blocked (service throws) (13 ms)
  ✓ 5) GET /api/ticket/:id -> 200 admin can view (14 ms)
  ✓ 6) GET /api/ticket/booking/:bookingId -> 401 without token (12 ms)
  ✓ 7) GET /api/ticket/booking/:bookingId -> 200 owner gets tickets (13 ms)
  ✓ 8) GET /api/ticket/booking/:bookingId -> 400 non-owner blocked (12 ms)
  ✓ 9) GET /api/ticket/booking/:bookingId -> 200 admin can view (14 ms)
  ✓ 10) POST /api/ticket/scan -> 401 without token (17 ms)
  ✓ 11) POST /api/ticket/scan -> 403 if not Business (businessOnly) (11 ms)
  ✓ 12) POST /api/ticket/scan -> 400 invalid DTO (13 ms)
  ✓ 13) POST /api/ticket/scan -> 400 QR not found (25 ms)
  ✓ 14) POST /api/ticket/scan -> 200 success marks ticket used (28 ms)
  ✓ 15) POST /api/ticket/scan -> 400 already used (10 ms)
  ✓ 16) POST /api/ticket/scan -> 400 expired token path sets status expired (17 ms)
  ✓ 17) PATCH /api/ticket/void/:id -> 401 without token (7 ms)
  ✓ 18) PATCH /api/ticket/void/:id -> 400 non-owner blocked (34 ms)
  ✓ 19) PATCH /api/ticket/void/:id -> 200 owner voids issued ticket (19 ms)
  ✓ 20) PATCH /api/ticket/void/:id -> 400 cannot void used ticket (13 ms)
  ✓ 21) PATCH /api/ticket/void/:id -> 400 already void (10 ms)

Test Suites: 1 passed, 1 total
Tests:       21 passed, 21 total
Snapshots:   0 total
Time:        3.42 s

```

Figure 36: Ticket Integration Test

```
Running one project: unit
PASS unit src/_tests_/unit/ticket/ticket.service.test.ts
TicketService Unit Tests
  ✓ getTicketById: throws if ticket not found (6 ms)
  ✓ getTicketById: throws if not owner and not privileged (1 ms)
  ✓ getTicketById: returns sanitized ticket for owner (1 ms)
  ✓ getTicketById: admin can view
  ✓ getTicketsByBooking: returns [] if none (1 ms)
  ✓ getTicketsByBooking: throws if not allowed (1 ms)
  ✓ getTicketsByBooking: returns sanitized for owner
  ✓ scanTicket: throws if qr not found
  ✓ scanTicket: throws if already used (1 ms)
  ✓ scanTicket: expires if expiresAt passed (updates status expired) (1 ms)
  ✓ scanTicket: marks ticket used
  ✓ voidTicket: throws if not found (1 ms)
  ✓ voidTicket: throws if not allowed (1 ms)
  ✓ voidTicket: cannot void used (1 ms)
  ✓ voidTicket: owner voids issued (1 ms)

Test Suites: 1 passed, 1 total
Tests:      15 passed, 15 total
Snapshots:  0 total
Time:       2.153 s
Ran all test suites matching ticket.service.test.ts.
```

Figure 37: Ticket Unit Test

```
PASS integration src/_tests_/integration/trip/trip.test.ts
Trip Integration Tests
  ✓ 1) POST /api/trip/create -> 401 without token (152 ms)
  ✓ 2) POST /api/trip/create -> 403 if role is not Business (businessOnly) (90 ms)
  ✓ 3) POST /api/trip/create -> 400 invalid DTO (34 ms)
  ✓ 4) POST /api/trip/create -> 500 Business not found (token id not in DB) (38 ms)
  ✓ 5) POST /api/trip/create -> 500 business not approved yet (31 ms)
  ✓ 6) POST /api/trip/create -> 201 success (34 ms)
  ✓ 7) GET /api/trip/:id -> 200 returns trip (33 ms)
  ✓ 8) GET /api/trip/:id -> 404 not found (33 ms)
  ✓ 9) GET /api/trip/business/mine -> 200 returns trips for business (36 ms)
  ✓ 10) PATCH /api/trip/edit/:id -> 400 if not owner business (34 ms)
  ✓ 11) PATCH /api/trip/edit/:id -> 400 totalSeats cannot be less than booked seats (30 ms)
  ✓ 12) PATCH /api/trip/edit/:id -> 200 success updates trip (35 ms)
  ✓ 13) DELETE /api/trip/delete/:id -> 400 not owner (30 ms)
  ✓ 14) DELETE /api/trip/delete/:id -> 200 success deletes (34 ms)
  ✓ 15) GET /api/trip/search -> 200 returns paginated search results (59 ms)

Test Suites: 1 passed, 1 total
Tests:      15 passed, 15 total
Snapshots:  0 total
Time:       3.769 s
```

Figure 38: Trip Integration Test

```

Running one project: unit
PASS unit src/__tests__/unit/trip/trip.service.test.ts
TripService Unit Tests
  ✓ 1) createTrip -> throws if business not found (13 ms)
  ✓ 2) createTrip -> throws if business not approved/verified (2 ms)
  ✓ 3) createTrip -> success sets availableSeats = totalSeats and trims from/to (2 ms)
  ✓ 4) getTripById -> throws if not found (1 ms)
  ✓ 5) getTripById -> returns sanitized trip (1 ms)
  ✓ 6) getTripsByBusiness -> calls repo with correct skip/limit and returns sanitized array (4 ms)
  ✓ 7) searchTrips -> builds filter and returns pagination fields (2 ms)
  ✓ 8) updateTrip -> throws if trip not found (1 ms)
  ✓ 9) updateTrip -> throws if business is not owner (1 ms)
  ✓ 10) updateTrip -> recalculates availableSeats if totalSeats changes (1 ms)
  ✓ 11) updateTrip -> throws if totalSeats < bookedSeats (1 ms)
  ✓ 12) updateTrip -> throws if update failed (repo returns null) (1 ms)
  ✓ 13) deleteTrip -> throws if trip not found (1 ms)
  ✓ 14) deleteTrip -> throws if not owner (1 ms)
  ✓ 15) deleteTrip -> success deletes and returns message (1 ms)

Test Suites: 1 passed, 1 total
Tests:       15 passed, 15 total
Snapshots:   0 total
Time:        1.281 s, estimated 3 s
Ran all test suites matching trip.service.test.ts.

```

Figure 39: Trip Unit Test

```

PASS integration src/__tests__/integration/user/user.test.ts (6.253 s)
User Integration Tests
  POST /api/user/register
    ✓ registers a user successfully (200) (169 ms)
    ✓ fails registration when body invalid (400) (10 ms)
    ✓ fails registration when phoneNumber already exists (500) (20 ms)
  POST /api/user/login
    ✓ fails login when DTO invalid (401) (12 ms)
    ✓ logs in successfully (201) and returns token (84 ms)
    ✓ fails login when password is wrong (500 in current controller) (76 ms)
    ✓ fails login when user doesn't exist (500 in current controller) (17 ms)
  GET /api/user/me
    ✓ rejects request without auth token (401) (11 ms)
    ✓ returns profile with valid token (200) (97 ms)

Test Suites: 1 passed, 1 total
Tests:       9 passed, 9 total
Snapshots:   0 total
Time:        6.372 s
Ran all test suites matching user.test.ts.

```

Figure 40: User Integration test

```

PASS unit src/_tests_/unit/user/user.service.test.ts
UserService unit tests
  ✓ createUser: throws if phoneNumber already exists (8 ms)
  ✓ createUser: hashes password, lowercases email, returns sanitized user (9 ms)
  ✓ loginUser: throws Invalid credentials if user not found by email (2 ms)
  ✓ loginUser: throws Authentication failed if getUserWithPassword returns null (2 ms)
  ✓ loginUser: throws Invalid credentials when password mismatch (3 ms)
  ✓ loginUser: returns token and sanitized user on success (4 ms)
  ✓ updateUser: throws User not found when user doesn't exist (1 ms)
  ✓ updateUser: throws collision error when phoneNumber is used by another user (1 ms)
  ✓ updateUser: throws if repository update returns null
  ✓ updateUser: returns sanitized updated user on success (1 ms)
  ✓ getAllUsers: calls repo with skip/limit and sanitizes results
  ✓ getUserById: throws User not found if repo returns null (1 ms)
  ✓ deleteUser: throws User not found if user does not exist (1 ms)
  ✓ deleteUser: calls repo.deleteUser on success (2 ms)
  ✓ sendResetPasswordEmail: throws HttpError 400 when email missing (2 ms)
  ✓ sendResetPasswordEmail: throws HttpError 404 when user not found (1 ms)
  ✓ sendResetPasswordEmail: sends email with reset link (1 ms)
  ✓ resetPassword: throws HttpError(400) when token or password missing
  ✓ resetPassword: throws HttpError(400) Invalid or expired token when jwt.verify fails
  ✓ resetPassword: updates password when token valid

Test Suites: 1 passed, 1 total
Tests:       20 passed, 20 total
Snapshots:   0 total
Time:        2.601 s
Ran all test suites.

```

Figure 41: User Unit Test

```

Test Suites: 6 passed, 6 total
Tests:       83 passed, 83 total
Snapshots:   0 total
Time:        6.027 s, estimated 14 s
Ran all test suites.

```

Figure 42: Integration Test Log

```

Test Suites: 6 passed, 6 total
Tests:       79 passed, 79 total
Snapshots:   0 total
Time:        4.46 s
Ran all test suites.

```

Figure 43: Unit Test Log

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	88.99	80.58	79.47	88.99	
src	100	100	100	100	
app.ts	100	100	100	100	
src/config	68	100	0	68	
email.ts	68	100	0	68	18-25
src/errors	100	100	100	100	
error.ts	100	100	100	100	
src/features/admin/controller	63.88	45	85.71	63.88	
admin.controller.ts	63.88	45	85.71	63.88	21-25,32-36,55-59,62-66,82-87,104-108,114-118,124-128,132-142
src/features/admin/repository	88.88	100	66.66	88.88	
admin.repository.ts	88.88	100	66.66	88.88	28-29,34-35
src/features/admin/routes	100	100	100	100	
admin.route.ts	100	100	100	100	
src/features/admin/service	96.29	93.75	100	96.29	
admin.service.ts	96.29	93.75	100	96.29	37-38
src/features/booking/controllers	76.72	73.33	83.33	76.72	
booking.controller.ts	76.72	73.33	83.33	76.72	66-70,74-90,110-114
src/features/booking/dtos	100	100	100	100	
booking.dto.ts	100	100	100	100	
src/features/booking/models	100	100	100	100	
booking.model.ts	100	100	100	100	
src/features/booking/repositories	86.66	100	80	86.66	
booking.repository.ts	86.66	100	80	86.66	33-38
src/features/booking/routes	100	100	100	100	
booking.route.ts	100	100	100	100	
src/features/booking/services	95.34	75	87.5	95.34	
booking.service.ts	95.34	75	87.5	95.34	72-73,166-167,180-183,190-191
src/features/booking/types	100	100	100	100	
booking.types.ts	100	100	100	100	
src/features/business/controllers	66.45	73.91	62.5	66.45	
business.controller.ts	66.45	73.91	62.5	66.45	47-53,63-66,83-84,93-94,108,113-114,118-127,131-146,150-159
src/features/business/dtos	100	100	100	100	
business.dto.ts	100	100	100	100	
src/features/business/models	100	100	100	100	
business.model.ts	100	100	100	100	
src/features/business/repositories	71.42	100	50	71.42	
business.repository.ts	71.42	100	50	71.42	22-26,33-34,46-54,57-58

Figure 44: Test Coverage Snippet